

Graph

Question No:1

Question description

Teddy is visiting the country wonderland. wonderland has n cities and m bi-directional roads. There are k types of tokens. Token i costs c_i . The costs of the tokens are such that for all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$. For each road, you need to have a particular set of tokens, if you want to travel it. Note that you don't have to give the tokens, you just need to show them. Thus, one token can be used at any number of roads, where it is required. Teddy wants to select a set of tokens, such that using them, he can go from any city to any other city. You have to help him minimize the total cost of tokens he buys.

Constraints

$$1 \leq n \leq 10^5$$

$$1 \leq m \leq 10^5$$

No road connects a city to the same city. However, there can be multiple roads between two cities.

$$1 \leq c_i \leq 10^{18}$$

For all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$

Input:

- The first line contains three space separated integers, n m and k .
- The second line contains k space separated integers, where the i^{th} integer denotes the price of i^{th} token, i.e. c_i .
- i^{th} of the next m lines contains three integers u_i, v_i, l_i , where l_i is the number of tokens required by the i^{th} road, followed by l_i indices denoting the tokens required. This road connects cities u_i and v_i .

Output:

- Print one integer containing the minimum cost of tokens Teddy has to buy, such that he can travel from any city to any other city. If it is impossible to choose such a set of tokens, print -1 .

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
```

EXPECTED OUTPUT

8

Test Case 2

INPUT (STDIN)

```
3 3 4
11 21 15 10
1 21 2 11 2
1 13 1 3
2 31 1 4
```

EXPECTED OUTPUT

57

Code:

```
1  #include <stdio.h>
2  #include <stdint.h>
3
4  #define MAXN 100005
5  #define MAXM 100005
6  #define MAXK 64
7
8  typedef long long ll;
9  typedef unsigned long long ull;
10
11  typedef struct {
12      int u, v;
13      ull mask;
14  } Road;
15
16  Road roads[MAXM];
17  ll cost[MAXK];
18
19  int parent[MAXN];
20
21  /* DSU Find */
22  int find(int x)
23  {
24      if (parent[x] == x)
25          return x;
26
27      return parent[x] = find(parent[x]);
28  }
```

```

29
30  /* DSU Union */
31  void unite(int a, int b)
32  {
33      a = find(a);
34      b = find(b);
35
36      if (a != b)
37          parent[b] = a;
38  }
39
40  /*
41      blocked = tokens that Teddy does NOT buy.
42
43      A road can be used only if none of its required
44      tokens are blocked.
45  */
46  int isConnected(ull blocked, int n, int m)
47  {
48      int components = n;
49
50      for (int i = 1; i <= n; i++)
51          parent[i] = i;
52
53      for (int i = 0; i < m; i++)
54      {

```

```

55         /*
56          * If the road requires any token that is blocked,
57          * we cannot use this road.
58          */
59         if ((roads[i].mask & blocked) != 0)
60             continue;
61
62         int u = find(roads[i].u);
63         int v = find(roads[i].v);
64
65         if (u != v)
66         {
67             unite(u, v);
68             components--;
69
70             if (components == 1)
71                 return 1;
72         }
73     }
74
75     return 0;
76 }
77
78 int main()
79 {
80     int n, m, k;
81
82     scanf("%d %d %d", &n, &m, &k);
83
84     /* Read token costs */
85     for (int i = 0; i < k; i++)
86     {
87         scanf("%lld", &cost[i]);
88     }
89
90     /* Read roads */
91     for (int i = 0; i < m; i++)
92     {
93         int u, v, l;
94
95         scanf("%d %d %d", &u, &v, &l);
96
97         roads[i].u = u;
98         roads[i].v = v;
99         roads[i].mask = 0;
100
101         for (int j = 0; j < l; j++)
102         {

```

```

103     int token;
104     scanf("%d", &token);
105
106     /*
107      * Token numbers are 1-based.
108      * Convert them to bit positions 0-based.
109      */
110     roads[i].mask |= (1ULL << (token - 1));
111 }
112 }
113
114 /*
115  * First check whether the graph is connected
116  * when Teddy buys ALL tokens.
117  */
118 if (!isConnected(0, n, m))
119 {
120     printf("-1\n");
121     return 0;
122 }
123
124 /*
125  * rejected = tokens that we decide NOT to buy.

```

```

126
127     Start from the most expensive token and try
128     removing it.
129
130     If the graph remains connected, we don't need
131     that expensive token.
132
133     Otherwise, we must buy it.
134 */
135 ull rejected = 0;
136 ll answer = 0;
137
138 for (int i = k - 1; i >= 0; i--)
139 {
140     ull currentToken = (1ULL << i);
141
142     /*
143     Try not buying this token.
144     */
145     if (isConnected(rejected | currentToken, n, m))
146     {
147         /*
148         Still connected, so we can reject it.
149         */
150         rejected |= currentToken;
151     }
152     else
153     {
154         /*
155         Removing it disconnects the graph,
156         so we must buy it.
157         */
158         answer += cost[i];
159     }
160 }
161
162 printf("%lld\n", answer);
163
164 return 0;
165 }

```

Output:

Test Case-1:

```
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
8
```

Test Case-2:

```
CE \graph.exe
3 3 4
11 21 15 10
1 21 2 11 2
1 13 1 3
2 31 1 4
57
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\DSA Practice>
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236